Task 3 - Recommender System Models

1. Approach

This task involved building and comparing three recommender system models on implicit feedback interaction data from interactions.csv. The setup treated observed user-item interactions as positive examples and sampled unobserved pairs as negatives, framing the task as binary classification.

The three models were:

- Matrix Factorization (MF) — dot product of learned embeddings
- MLP-Based Recommender — concatenated embeddings passed through fully connected layers
- Neural Collaborative Filtering (NCF) — combining both MF and MLP signals

2. Key Design Choices

Negative Sampling

Since the dataset only contained positive interactions, negative samples were generated for each observed (user, item) pair. A set of all known interactions was built upfront for O(1) lookup. The num_neg parameter was set to 15 for initial training.

Model Architectures

MF used nn.Embedding layers for both users and items, with a dot product as the interaction score. This is the simplest possible baseline.

MLP concatenated user and item embeddings and passed them through three linear layers (64 -> 32 > 1) with ReLU activations. I deliberately kept this shallow to **avoid overfitting** given the dataset size.

NCF computed both an MF-style dot product score and an MLP score independently, then fused them through a small fusion network (Linear(2,16) -> ReLU -> Linear(16,1)). Embeddings were shared between both paths.

Setting	Value
Loss function	BCEWithLogitsLoss
Optimizer	Adam (lr = 0.001)
Batch size	256
MF embedding dim	1024
MLP / NCF embedding dim	128

MF epochs	40
MLP / NCF epochs	200

3. Results

Models were evaluated using Hit@10.

MF served as a strong baseline despite its simplicity, benefiting from the high embedding dimension (1024) which gave it significant capacity. The MLP showed improved performance due to its ability to capture non-linear user-item interactions. NCF combined both signals and generally matched or exceeded individual models.

4. What Was Tried Beyond Basics

NCF Architecture

The bonus NCF model was implemented by computing MF and MLP scores separately and fusing them through a learned fusion network rather than simple concatenation. This allowed the model to learn how to weight each signal adaptively.

Embedding Dimension Tuning

MF was run with emb_dim=1024 which gave it more capacity to compensate for the simplicity of the dot product interaction. MLP and NCF used emb_dim=128, which I thought was sufficient given the additional depth of those architectures. Higher dims on MLP tended to overfit.

MLP Depth

The MLP was kept at two hidden layers (64 -> 32). Deeper networks with 512 units, for example, showed no improvement and trained significantly slower, suggesting the dataset did not require that level of capacity.

5. Observations

- MF only computes a **dot product** - it assumes user and item relationships are purely linear. MLP can learn **non-linear combinations** of features, meaning it can capture more complex preference patterns.
- MF might outperform MLP in cases where the data is **sparse;** fewer interactions means less signal for the MLP to learn from, so its extra complexity doesn't help. MF also tends to win when the underlying preference structure genuinely is linear or when you have very few training epochs since MF converges faster.
- BCEWithLogitsLoss is the right choice here - it applies sigmoid internally and is numerically stable, avoiding the need to manually sigmoid model outputs.